



Automating TOTAL with AI

A Project Prometheus white paper — how an AI agent can read and write TOTAL appraisal reports. Educational reference only.

[← All tools](#)

A Project Prometheus white paper. Published by The Appraiser Coach. Educational reference — not a build guide, not a product release.

Read this first — disclaimer

- **This is a white paper, not a software release.** No source code is published here. No compiled binaries are distributed. No installer exists. There is no "download and run" path.
- **No warranty, no tech support.** If you decide to attempt something like what is described here, you do so entirely at your own risk and responsibility. The Appraiser Coach, Dustin Harris, and any contributors mentioned are not your engineers and will not troubleshoot your build.
- **You must remain in compliance with every applicable software license.** That includes — but is not limited to — a la mode's TOTAL end-user license agreement, Microsoft .NET licensing, Python's license, and any AI provider's terms of service. If you don't have a legally licensed copy of TOTAL on your own machine, this document is purely academic for you. The recommended commercial integration path is a la mode's [TOTAL Store / VIP API partner program](#); pursue that if you are building a commercial product.
- **Nothing here is legal advice.** Consult an attorney about your specific situation before acting on anything described in this document.

Contents

1. The problem we tried to solve
2. Path 1 — the screen-reader approach
3. Path 2 — direct `.zap` read/write
4. What this enabled
5. High-level architecture outline

6. What did not work

7. Take-aways for the appraiser community

1. The problem we tried to solve

Residential appraisal in the United States is overwhelmingly written in one piece of software: a la mode's TOTAL. TOTAL is mature, feature-rich, and the de-facto standard for over 100,000 working appraisers. It is also, from the perspective of an appraiser who wants to add AI automation to their workflow, a closed black box. There is no public API. The published file format (`.zap`) is encrypted. The recommended integration path — TOTAL Store / VIP — requires a vendor partnership and an access token that individual appraisers cannot obtain.

This created a hard wall for Project Prometheus. The thesis behind Prometheus was that an AI agent could take over the mechanical portions of an appraisal workflow — order intake, comp research, field-mapping, pre-fill of report shells, quality control — leaving the appraiser to focus on the parts that require human judgment. Most of those mechanical tasks are about getting data *into* a TOTAL report. With no way to programmatically write to a TOTAL report, the rest of the system was moot. We had to solve the TOTAL-write problem before anything else mattered.

We tried two approaches. The first was the obvious one. It worked, technically, and we abandoned it. The second is the one we kept.

2. Path 1 — the screen-reader approach

The first approach was visual UI automation. The idea was straightforward: run TOTAL on a Windows machine the way a human runs it, and have an AI agent operate the user interface using the same affordances a human would — keyboard, mouse, screen reading. There is decent open-source tooling for this: `pywinauto` for Win32 control-level interaction, `pyautogui` for pixel-and-coordinate fallback, and various agent frameworks that connect an LLM to those tools and a screenshot of the live window. The agent looks at the screen, decides what to click, clicks it, types the value, and verifies visually that it landed.

It worked. It also did not scale. The limits showed up quickly:

- **Fragility.** Any TOTAL UI change — even between minor versions — broke our scripts. A repositioned button or a renamed field meant the agent could no longer find

what it was looking for. We were one a la mode update away from rebuilding everything.

- **Speed.** Each field interaction took multiple seconds: focus the window, locate the field, click, wait for it to accept focus, type the value, verify, move on. A 1004 report has 130+ fillable fields. A full pre-fill took close to an hour in the best case.
- **Hallucinations.** A vision-capable agent occasionally clicked the wrong tab, picked the wrong field, or typed a value into the comp 2 column when it was supposed to go into comp 1. Even a 5% error rate is unacceptable when each error becomes a USPAP-relevant defect that the appraiser then has to find and fix.
- **Modal dialogs and focus loss.** TOTAL pops up modal dialogs for many actions (save confirmation, contact-picker, MISMO validation). Each of these is a state the script has to anticipate and handle. We accumulated a long list of dialog handlers, and we kept finding new ones.
- **Verifiability.** Even after the agent claimed to fill a field, the only way to confirm it actually saved was to re-read it visually. That doubled the per-field time and added another opportunity for misreading.
- **Resource intensity.** Running TOTAL in a Windows VM with a full graphical session, kept alive 24/7 so the agent could connect, is expensive in both compute and operator attention. The VM crashed. Chrome's CDP tunnel dropped. TOTAL hung. Each failure was its own debug session.

The screen-reader path is a reasonable fallback for tasks that genuinely require the UI — flows that hit dialog boxes the data layer can't replicate, MFA portal uploads, the "Send to Vendor" trigger if you're integrating with TOTAL Store via the VIP API later. For everything else, we needed a path that did not require the UI to be involved at all.

3. Path 2 — direct `.zap` read/write

The breakthrough was realizing that TOTAL's `.zap` file format, while encrypted, is fundamentally just a ZIP archive with a known encryption key — and the bundled .NET assemblies inside any licensed TOTAL installation already contain the code that handles both the unzip and the rezip. Doug, who runs the IT for our operation, discovered that you can call those bundled assemblies directly from a small C# program, without TOTAL itself being open, without the encryption key ever appearing in our code, and without reverse-engineering anything proprietary.

The specific pieces involved are TOTAL's own .NET classes:

- **Reading:** `alamode.Appraisal.Business.ReportData.ZipReader` provides methods like `GetXmlDocument("ReportData.XML")`, `ExtractFile`, `ExtractFiles`, `HasFile`, and `GetComment`. These work password-less because TOTAL's native zip library — `dznscore.dll` — auto-applies the universal encryption key transparently when called from inside a TOTAL installation. The key is never in our code. We never extract it, copy it, or distribute it.
- **Writing:** `alamode.Common.Library.Zip.ZipAllFoldersAndFiles(folderPath, comment, password=null, zipFilename, excludeExtension)` produces a valid encrypted `.zap` file that TOTAL itself opens cleanly. We verified the round-trip end-to-end: extract a report, modify its `ReportData.XML`, repack, open in TOTAL, all 4,681 field nodes intact, marker we added still present.

The conceptual model: a `.zap` is a directory tree containing an XML file (`ReportData.XML`), a metadata file (`RepInfo.xml`), and a workfile folder with embedded attachments. The XML lists every form field on every page of the report. Setting a field is a matter of finding the right `FieldName` in the XML and editing its `EditContent` text node. Clone a template, set the fields, save the file, done. TOTAL opens the result as a normal report.

This approach has none of the drawbacks of the screen-reader path:

- **Speed.** Setting all 130 fields takes well under a second.
- **Verifiability.** You can read every field back after writing it. The agent's success criteria are unambiguous.
- **No UI dependency.** No Windows GUI session, no TOTAL running, no modal dialogs. Just file I/O on the user's machine.
- **Robustness across TOTAL versions.** The XML schema for fields has been stable across multiple major versions of TOTAL because TOTAL itself depends on it. UI layouts change; field names don't.
- **No reverse engineering.** Every API call is a documented .NET method on assemblies that ship with a legally-licensed TOTAL installation, callable through normal .NET reflection.

The compliance posture, in plain language

This is a back door in the sense that it is not the integration path a la mode advertises. It is not a back door in the sense of breaking, bypassing, or evading any technical protection mechanism. The encryption key stays inside TOTAL's own code. The DLLs being called are on the user's own machine, installed by the user's own legitimately

licensed copy of TOTAL. The data being read and written is the user's own appraisal data. No part of this approach copies, redistributes, decompiles, or reverse-engineers TOTAL itself.

It is, in principle, similar to writing an Excel macro: the macro doesn't ship with Excel, but it operates inside a legally-installed Excel using Excel's own object model. Whether a la mode's specific EULA permits this kind of programmatic use of bundled assemblies is a question for an attorney and a la mode's licensing team — we are *not* publishing a definitive answer on that. We are publishing this white paper because the technical pattern is interesting and educational, and because appraisers who want to pursue serious AI integration should know it exists and what the trade-offs are.

4. What this enabled

Once read and write at the `.zap` file layer were both solved, the rest of the Prometheus pipeline became tractable:

- **Engagement letter to TOTAL in seconds.** An AI agent parses the engagement letter PDF into a structured object, then maps that object directly to TOTAL field names and writes the report shell. What used to take five to ten minutes of typing happens in under a second.
- **Round-trip verification.** Every write is followed by a read-back. If a field didn't land where it was supposed to, the agent retries or escalates immediately, before the appraiser sees a broken report.
- **Workfile embedding.** Research files, GIS maps, the original engagement letter, and any other supporting documents can be embedded directly into the `.zap`'s Workfile panel — exactly where TOTAL expects them — so the appraiser doesn't have to leave TOTAL to find context.
- **Pre-inspection report shell.** By the time the appraiser arrives at the property for inspection, the report shell is already prefilled with everything the engagement letter and county records provided. The appraiser focuses on the parts of the work that require human judgment.
- **Pipeline integration.** All of this hooks into an AI agent ecosystem (Claude Code, OpenAI's agent SDKs, OpenClaw, or any framework that supports the Model Context Protocol) through a thin MCP server. The AI agent talks MCP; the MCP server talks to the C# tool; the C# tool talks to TOTAL's DLLs. Each layer is replaceable.

5. High-level architecture outline

This is an outline, not a build guide. If you want to attempt something like this, you (or your developer) need the following — at minimum:

1. **A licensed copy of TOTAL installed on the machine where the automation will run.** This is non-negotiable. The whole approach depends on TOTAL's own assemblies being present and operable. There is no "TOTAL-free" version of this.
2. **Microsoft .NET Framework 4.8 (x86)** and a working C# compiler (`csc.exe` ships with .NET).
3. **A small C# program (call it `zap-tool.exe` or whatever you like) that loads the relevant TOTAL assemblies via .NET reflection and exposes CLI commands** such as: `extract` , `repack` , `list-forms` , `list-fields` , `get-field` , `set-field` , `set-fields` , `clone-template` , `add-workfile` . Each command operates on a `.zap` file path. The implementation involves identifying the right `FieldName` in the extracted `ReportData.XML` , modifying the `EditContent` text node, and using the bundled `rezip` method to repack.
4. **A Python (or any language) wrapper that exposes those CLI commands as an MCP server.** The MCP standard ([Model Context Protocol](#)) defines how AI agents discover and call tools. The wrapper translates "MCP tool call from an AI agent" into "shell out to `zap-tool.exe` with the right arguments" and returns the result back to the agent as MCP output.
5. **An AI agent that can speak MCP.** Claude Code, OpenClaw, custom agents using the Anthropic or OpenAI SDKs — anything that supports MCP. Connect the MCP server, give the agent a task ("here's an engagement letter PDF; build the report shell from our 1004 template"), and watch.
6. **A safe operating posture.** Treat the AI agent as a junior assistant: every write is followed by a read-back, every action is logged, and humans verify before any external communication or report delivery. Project Prometheus runs a verification sweep after every build and refuses to mark an order complete if the verification fails.

Things to map out before writing any code

- **The field-name dictionary.** TOTAL uses internal names like `SUBPROPADDRESS` , `BORROWER1` , `BCDATA` , `GSU_AD_ADDR1` . Every form has hundreds of them, and they are not documented externally. You discover them by extracting a known report and reading the XML. Plan for several weeks of pinning field bindings across the forms you care about.

- **Cascade behavior.** Some fields propagate automatically through TOTAL's "linked-field" cascade; others do not. The Assignment Tab cascade in particular handles a lot of derivation. Some fields look the same but behave differently. You will run experiments.
- **Template hygiene.** Real-world TOTAL templates often ship with stale data baked in from a prior assignment (Hawaii FEMA zones, Kauai census tracts, an old borrower name). Scrub those before writing your own data, or they will reappear in the final report.
- **Form-specific differences.** 1004, 1004D, 1073, 1025, 2055, GPAR, and the new UAD 3.6 dynamic URAR each have different field schemas. The 2055 doesn't have a sales-comparison grid. The 1004D doesn't have a cost approach. Map per form.

6. What did not work

For completeness, the paths we tried and rejected:

Approach	Why we tried it	Why we rejected it
<code>alaReport.OpenReport(path)</code>	Looked like the obvious managed-code entry point — the constructor exists, the method signature is public.	Hard-crashes the process on a licensed installation. Requires a full TOTAL runtime environment we could not synthesize headlessly. Weeks of reverse-engineering risk to make work, with no obvious benefit over the simpler ZipReader/ZipAllFoldersAndFiles approach.
TOTAL Store / VIP COM API (unregistered)	This is a la mode's official partner-integration path. If we could use it, we would.	Returns "Access denied" without a vendor authorization token. Reversing or fabricating the token would clearly violate the TOS. Individual appraisers cannot obtain a token. This path is appropriate for commercial integration vendors, not appraiser-tooling experimentation.

Approach	Why we tried it	Why we rejected it
In-process injection into <code>WinTOTAL.exe</code>	Considered briefly as a way to get inside the running process.	Rejected on principle — it would constitute reverse engineering, would violate any reasonable interpretation of the EULA, and is fragile across TOTAL versions.
UI automation via <code>pywinauto</code> / <code>pyautogui</code>	The straightforward approach. Doesn't require any insight into TOTAL's internals.	Documented above (Path 1). Useful as a fallback for genuinely UI-only flows; not viable as the primary data-write mechanism.

7. Take-aways for the appraiser community

- **Programmatic TOTAL integration is possible without a vendor partnership** if you have a legally-licensed copy of TOTAL and you accept the constraints described above (you do the engineering, you stay compliant with the EULA, you accept the risk).
- **The official path — TOTAL Store / VIP API — exists and works.** Commercial vendors who want a long-term integration should pursue it. It is the right path for a software business.
- **For appraisers experimenting with AI to streamline their own practice**, the direct-`.zap` approach is what made Project Prometheus practical. We're documenting it here so the next person doesn't have to rediscover it from scratch.
- **UAD 3.6 (mandate November 2, 2026) will reshape this.** The dynamic URAR format and the new MISMO 3.6 standard change the file shape and the field schema. Anyone planning serious work here should track the UAD 3.6 transition closely and budget for a rebuild.
- **None of this replaces the appraiser.** Every concept in this paper is plumbing — fast plumbing — that connects an appraiser's judgment to the report. The appraiser is still the appraiser. AI is the assistant.

Repeat of the disclaimer

- This document describes a pattern, not a product. There is no software to download, no installer, no compiled binaries, no source code, no tech support.
- If you attempt to build something based on what's described here, you do so at your own risk. The Appraiser Coach, Dustin Harris, Doug, and anyone else mentioned in this document accept no responsibility for the outcome.
- You remain solely responsible for compliance with every applicable software license, including a la mode's TOTAL EULA, .NET licensing, Python licensing, and any AI provider's terms of service.
- Nothing in this document is legal advice. Consult an attorney about your own situation.
- If your intended use is commercial, the recommended path is a la mode's TOTAL Store partner program.

Published by **The Appraiser Coach** as part of Project Prometheus. For coaching, training, and the community building these workflows: theappraisercoach.com →

Educational reference. No source code, no compiled binaries, no warranty, no tech support. View this site's source on [GitHub](#).